Chapter 10

Asymmetric-Key Cryptography

10-1 INTRODUCTION

Symmetric and asymmetric-key cryptography will exist in parallel and continue to serve the community. We actually believe that they are complements of each other; the advantages of one can compensate for the disadvantages of the other.

Topics discussed in this section:

10.1.1 Keys
10.1.2 General Idea
10.1.3 Need for Both
10.1.4 Trapdoor One-Way Function
10.1.5 Knapsack Cryptosystem

10-1 INTRODUCTION

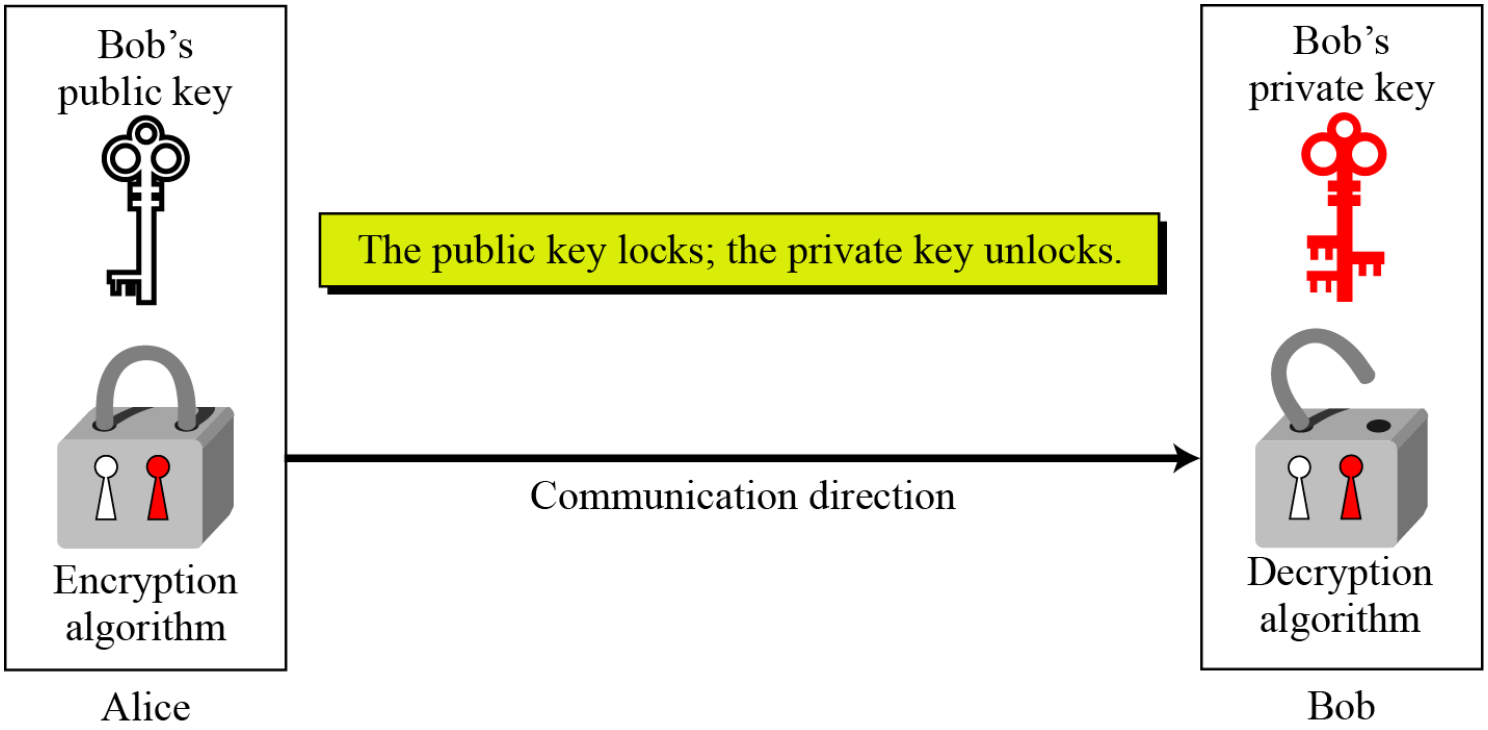
Note

Symmetric-key cryptography is based on sharing secrecy; asymmetric-key cryptography is based on personal secrecy.

10.1.1 Keys

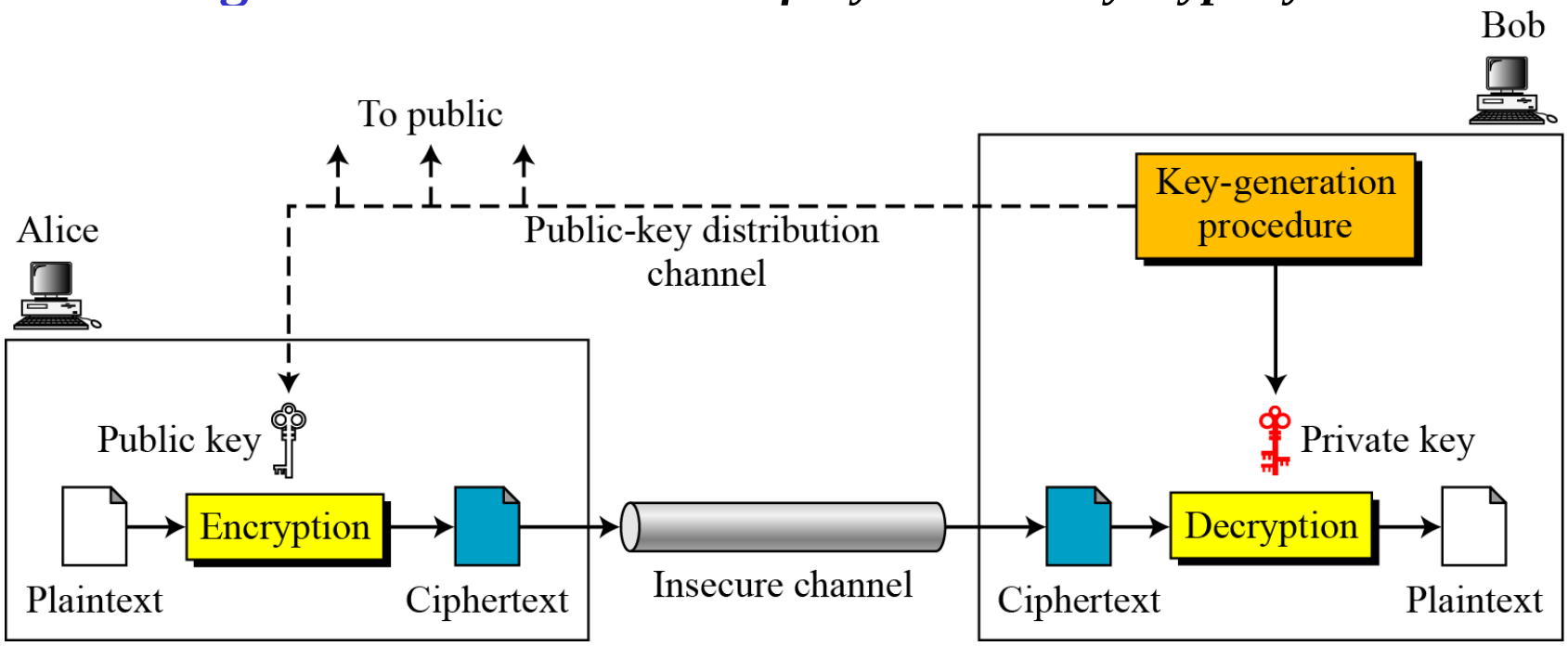
Asymmetric key cryptography uses two separate keys: one private and one public.

Figure 10.1 *Locking and unlocking in asymmetric-key cryptosystem*



10.1.2 General Idea

Figure 10.2 *General idea of asymmetric-key cryptosystem*



10.1.2 Continued

Plaintext/Ciphertext

Unlike in symmetric-key cryptography, plaintext and ciphertext are treated as integers in asymmetric-key cryptography.

Encryption/Decryption

$$C = f(K_{public}, P) \qquad P = g(K_{private}, C)$$

10.1.3 Need for Both

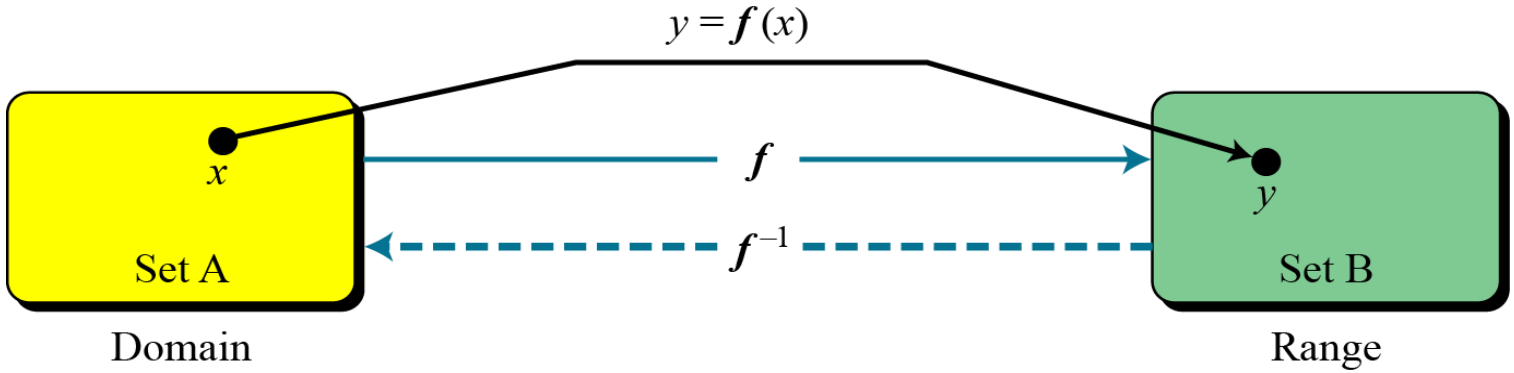
There is a very important fact that is sometimes misunderstood: The advent of asymmetric-key cryptography does not eliminate the need for symmetric-key cryptography.

10.1.4 Trapdoor One-Way Function

The main idea behind asymmetric-key cryptography is the concept of the trapdoor one-way function.

Functions

Figure 10.3 *A function as rule mapping a domain to a range*



10.1.4 Continued

One-Way Function (OWF)

1. f is easy to compute.

2. f^{-1} is difficult to compute.

Example 10. 1

When n is large, $n = p \times q$ is a one-way function. Given p and q , it is always easy to calculate n ; given n, it is very difficult to compute p and q. This is the factorization problem.

10.1.4 Continued

Trapdoor One-Way Function (TOWF)

3. Given y and a trapdoor(secret), x can be computed easily.

Example 10. 2

When n is large, the function $y = x^k \bmod n$
is a trapdoor one-way function.
Given x, k, and n, it is easy to calculate y.
Given y, k, and n, it is very difficult to calculate x.
This is the discrete logarithm problem.

However, if we know the trapdoor,
k′ such that $k \times k' = 1 \bmod \phi(n)$,
we can use $x = y^{k'} \bmod n$ to find x.

10-2 RSA CRYPTOSYSTEM

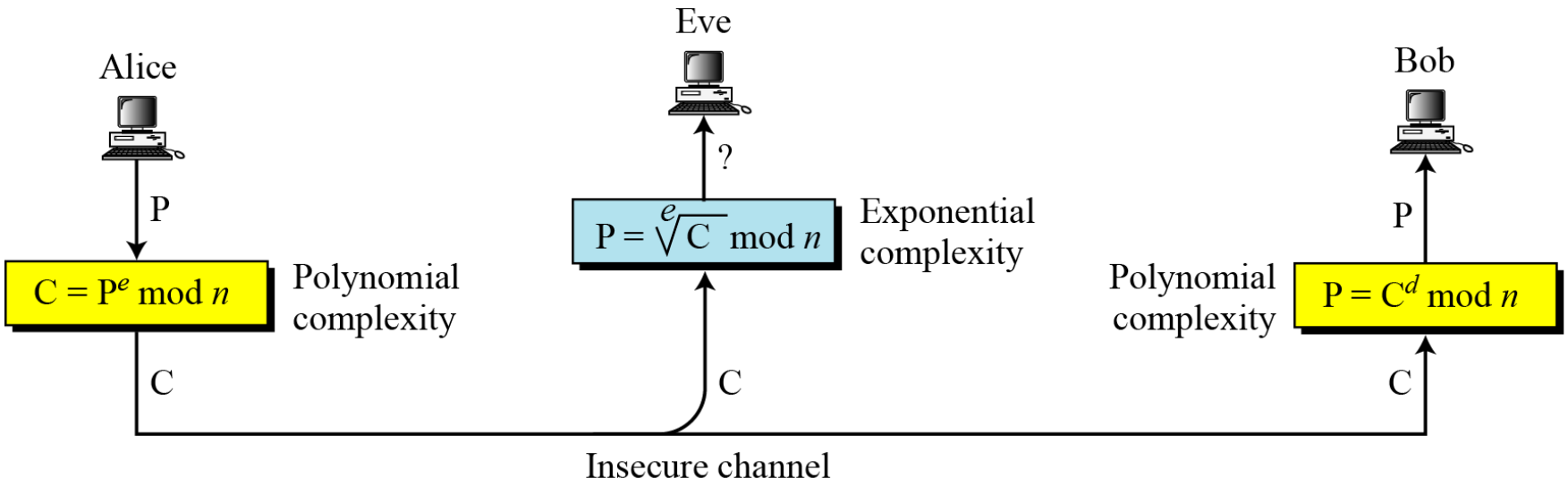
The most common public-key algorithm is the RSA cryptosystem, named for its inventors (Rivest, Shamir, and Adleman).

Topics discussed in this section:

10.2.1 Introduction
10.2.2 Procedure
10.2.3 Some Trivial Examples
10.2.4 Attacks on RSA
10.2.5 Recommendations
10.2.6 Optimal Asymmetric Encryption Padding (OAEP)
10.2.7 Applications

10.2.1 Introduction

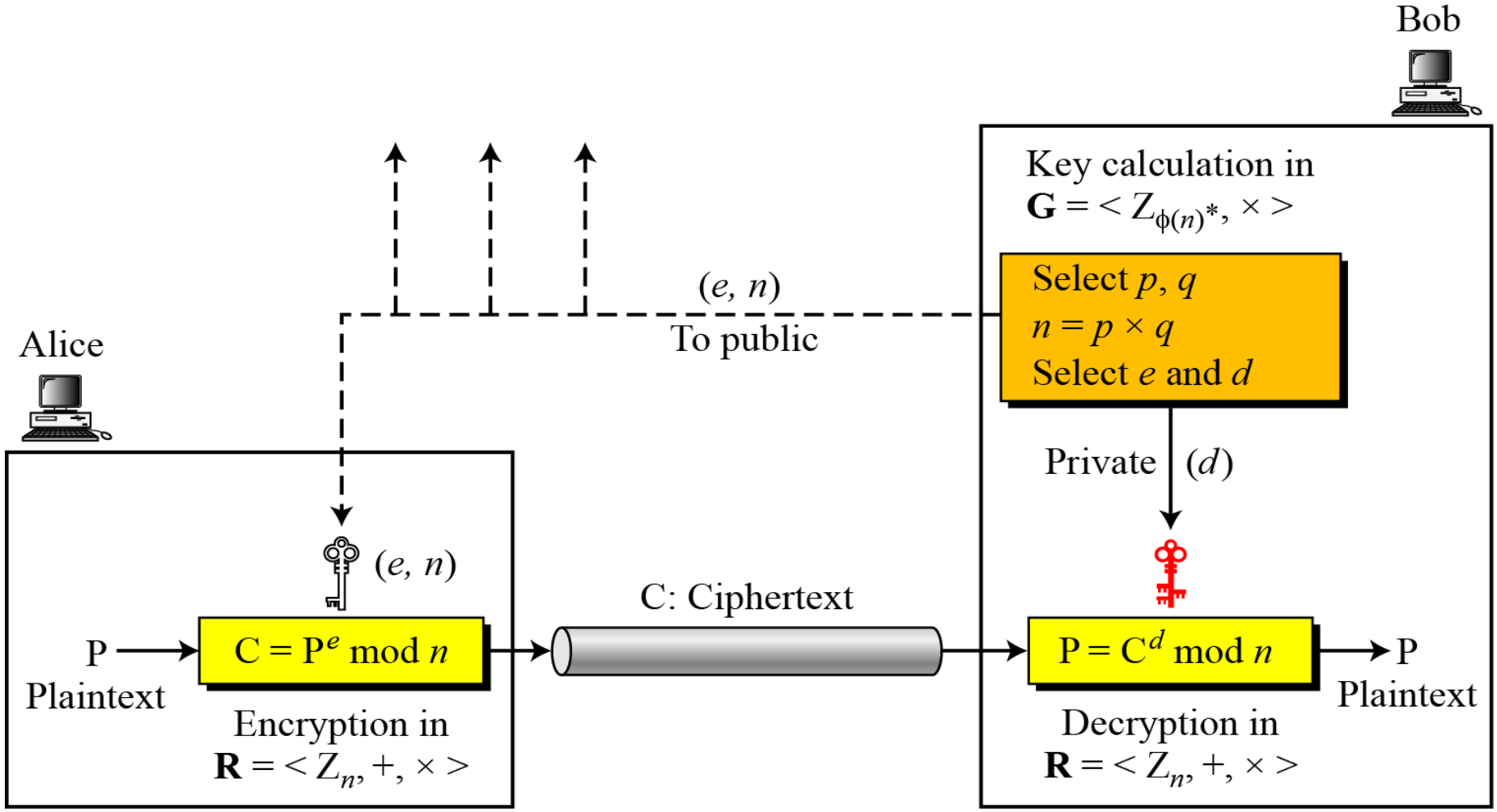
Figure 10.5 *Complexity of operations in RSA*



RSA uses modular exponentiation for encryption/decryption; To attack it, Eve needs to calculate $\sqrt[e]{C} \bmod n$.

10.2.2 Procedure

Figure 10.6 *Encryption, decryption, and key generation in RSA*



In RSA, the tuple (e, n) is the public key; the integer d is the private key.

10.2.2 Continued

Algorithm 10.2 *RSA Key Generation*

RSA_Key_Generation

{

Select two large primes p and q such that $p \neq q$.

$n \leftarrow p \times q$

$\phi(n) \leftarrow (p - 1) \times (q - 1)$

Select e such that $1 < e < \phi(n)$ and e is coprime to $\phi(n)$

$d \leftarrow e^{-1} \bmod \phi(n)$ // d is inverse of e modulo $\phi(n)$

Public_key $\leftarrow$ (e, n) // To be announced publicly

Private_key $\leftarrow d$ // To be kept secret

return Public_key and Private_key

}

Encryption

Algorithm 10.3 *RSA encryption*

```
RSA_Encryption (P, e, n)                    // P is the plaintext in Z_n and P < n
{
    C ← Fast_Exponentiation (P, e, n)       // Calculation of (P^e mod n)
    return C
}
```

In RSA, p and q must be at least 512 bits; n must be at least 1024 bits.

Decryption

Algorithm 10.4 *RSA decryption*

```
RSA_Decryption (C, d, n)                    //C is the ciphertext in Z_n
{
    P ← Fast_Exponentiation (C, d, n)       // Calculation of (C^d mod n)
    return P
}
```

10.2.2 Continued

Proof of RSA

If $n = p \times q$, $a < n$, and k is an integer, then $a^{k \times \phi(n)+1} \equiv a \pmod{n}$.

$P_1 = C^d \bmod n = (P^e \bmod n)^d \bmod n = P^{ed} \bmod n$

$ed = k\phi(n) + 1$ // d and e are inverses modulo $\phi(n)$

$P_1 = P^{ed} \bmod n \rightarrow P_1 = P^{k\phi(n)+1} \bmod n$

$P_1 = P^{k\phi(n)+1} \bmod n = P \bmod n$ // Euler's theorem (second version)

10.2.3 Some Trivial Examples

Example 10. 5

Bob chooses 7 and 11 as *p* and *q* and calculates *n* = 77. The value of $\phi(n) = (7 - 1)(11 - 1)$ or 60. Now he chooses two exponents, *e* and *d*, from $Z_{60}*$. If he chooses *e* to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$ (they are inverses of each Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5.

Plaintext: 5	$C = 5^{13} = 26 \bmod 77$	Ciphertext: 26

Bob receives the ciphertext 26 and uses the private key 37 to decipher the ciphertext:

Ciphertext: 26	$P = 26^{37} = 5 \bmod 77$	Plaintext: 5

10.2.3 Some Trivial Examples

Example 10. 6

Now assume that another person, John, wants to send a message to Bob. John can use the same public key announced by Bob (probably on his website), 13; John's plaintext is 63. John calculates the following:

Plaintext: 63	$C = 63^{13} = 28 \bmod 77$	Ciphertext: 28

Bob receives the ciphertext 28 and uses his private key 37 to decipher the ciphertext:

Ciphertext: 28	$P = 28^{37} = 63 \bmod 77$	Plaintext: 63

10.2.3 Some Trivial Examples

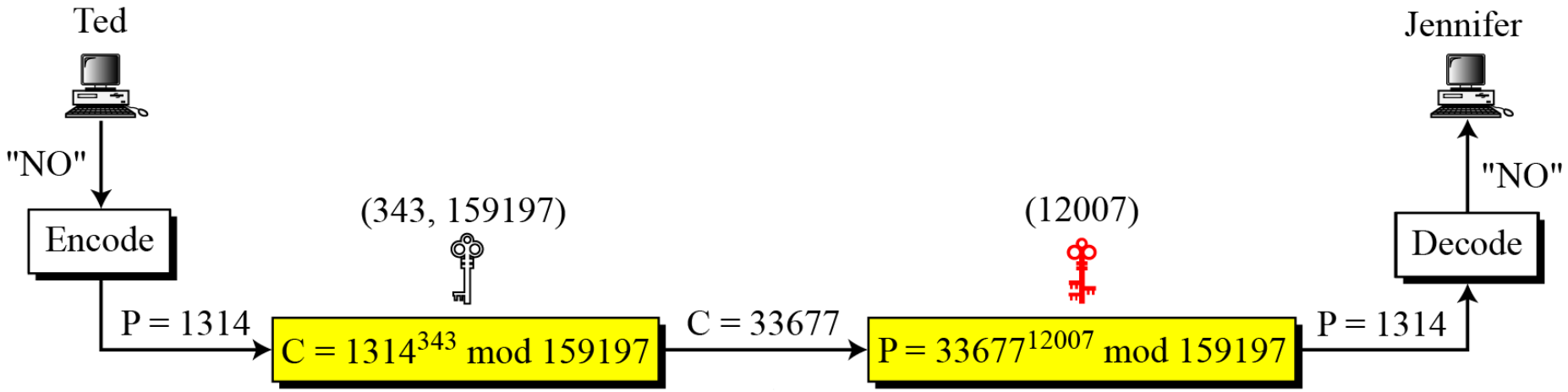
Example 10. 7

Jennifer creates a pair of keys for herself. She chooses p = 397 and q = 401. She calculates n = 159197. She then calculates ϕ(n) = 158400. She then chooses e = 343 and d = 12007. Show how Ted can send a message to Jennifer if he knows e and n.

Suppose Ted wants to send the message "NO" to Jennifer. He changes each character to a number (from 00 to 25), with each character coded as two digits. He then concatenates the two coded characters and gets a four-digit number. The plaintext is 1314. Figure 10.7 shows the process.

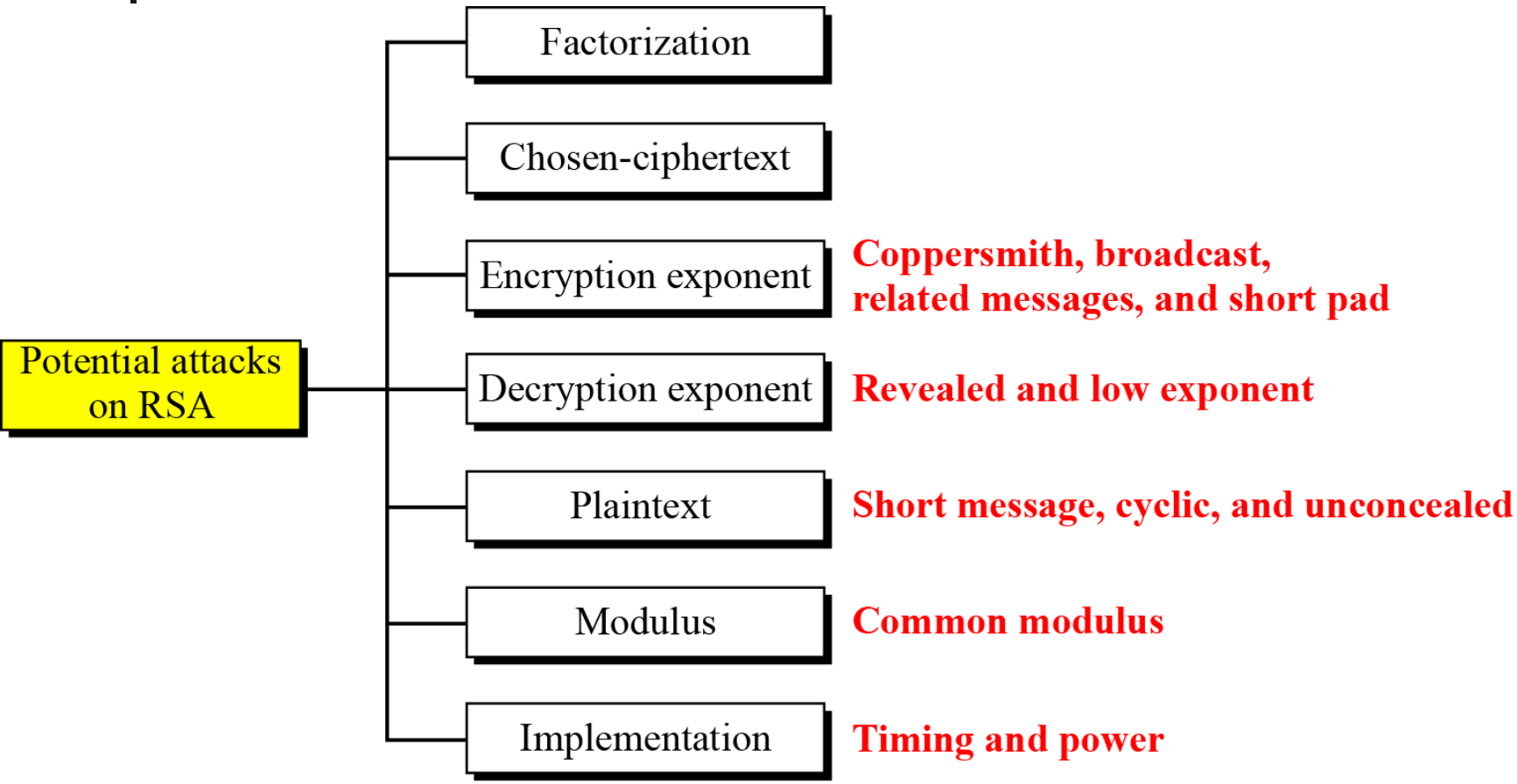
10.2.3 Continued

Figure 10.7 *Encryption and decryption in Example 10.7*



10.2.4 Attacks on RSA

Figure 10.8 *Taxonomy of potential attacks on RSA*



Coppersmith method for attacking **RSA** include cases when the public **exponent** e is small or when partial knowledge of the secret key is available.

10.2.6 Continued

Example 10. 8

Here is a more realistic example. We choose a 512-bit p and q, calculate n and $\phi(n)$, then choose e and test for relative primeness with $\phi(n)$. We then calculate d. Finally, we show the results of encryption and decryption. The integer p is a 159-digit number.

$p =$	96130345313583504574191581280615427909309845594996215822583150879647940455056470638491257160180347503120986666064924201918087806674210960633542199266612 09
$q =$	12060191957231446918276794204450896001555925054637033936061798321731482148483764659215389453209175225273226830107120695604602513887145524969000359660045617

10.2.6 Continued

Example 10. 8 *Continued*

The modulus $n = p \times q$. It has 309 digits.

$n =$

1159350417396761496889250986461588752377145737545414477548552613761478854083263508172768788159683251684688493006254857641112501624145523391829271625076567727274600970827141277304349605005563472745666280600999240371029914244722922157727985317270338393813346926841373276220009666766718318310883734208234443709 53

$\phi(n) = (p - 1)(q - 1)$ has 309 digits.

$\phi(n) =$

1159350417396761496889250986461588752377145737545414477548552613761478854083263508172768788159683251684688493006254857641112501624145523391829271625076567510542336084929167520344826279881175547876570139234444057169895817281960982263610754672118646121713591073586406140088851702653772772644673410662438576641 28

10.2.6 Continued

Example 10. 8 *Continued*

Bob chooses e = 35535 (the ideal is 65537) and tests it to make sure it is relatively prime with ϕ(n). He then finds the inverse of *e* modulo ϕ(n) and calls it *d*.

$e =$	35535
$d =$	58008302860037763936093661289677917594669062089650962180422866111380593852822358731706286910030021710859044338402170729869087600611530620252495988444804756824096624708148581713046324064407770483313401085094738529564507193677406119732655742423721761767462077637164207600337085333288532144708859551366702948 31

10.2.6 Continued

Example 10. 8 *Continued*

Alice wants to send the message "THIS IS A TEST", which can be changed to a numeric value using the 00–26 encoding scheme (26 is the space character).

P = 19070818260818260026190418 19

The ciphertext calculated by Alice is $C = P^e$, which is

C =
475309123646226827206365550610545180942371796070491716523923905 4
452960613199328566617843418359114151197411252005682979794571736036
101278218847892741566090480023507190715277185914975188465888632101
148354103361657898467968386763733765777465625079280521148141844048
141844308127730590046928742485591664621086 56

10.2.6 Continued

Example 10. 8 *Continued*

Bob can recover the plaintext from the ciphertext using $P = C^d$, which is

P =	1907081826081826002619041819

The recovered plaintext is "THIS IS A TEST" after decoding.

Problem:1.

Given, p=5, q=11, e=3, m=9.

Perform encryption and decryption

Problem:2.

Given, n=77, e=7, Find d ?

43

The ciphertext = 5, find the plaintext?

26

10-3 RABIN CRYPTOSYSTEM

The Rabin cryptosystem can be thought of as an RSA cryptosystem in which the value of e and d are fixed.

The encryption is

$$C \equiv P^2 \pmod{n}$$

and the decryption is

$$P \equiv C^{1/2} \pmod{n}.$$

Topics discussed in this section:

10.3.1 Procedure

10.3.2 Security of the Rabin System

- Computationally secure against a chosen plaintext attack
  - Provided that the modulus n = p.q can not be factored.
- n is the public key.
- The primes p and q are the private key.
- Choose to simplify the computation of square roots modulo p and q

B encrypts a message m and sends the ciphertext C to A

- Encryption:
  - Obtain A's public key n.
  - Represent the message as an integer m in the range {0, 1, . . . ,n-1}.
  - Compute $C \equiv P^2 \pmod{n}$
  - Send the ciphertext c to A

- A decrypts the ciphertext C as follows:

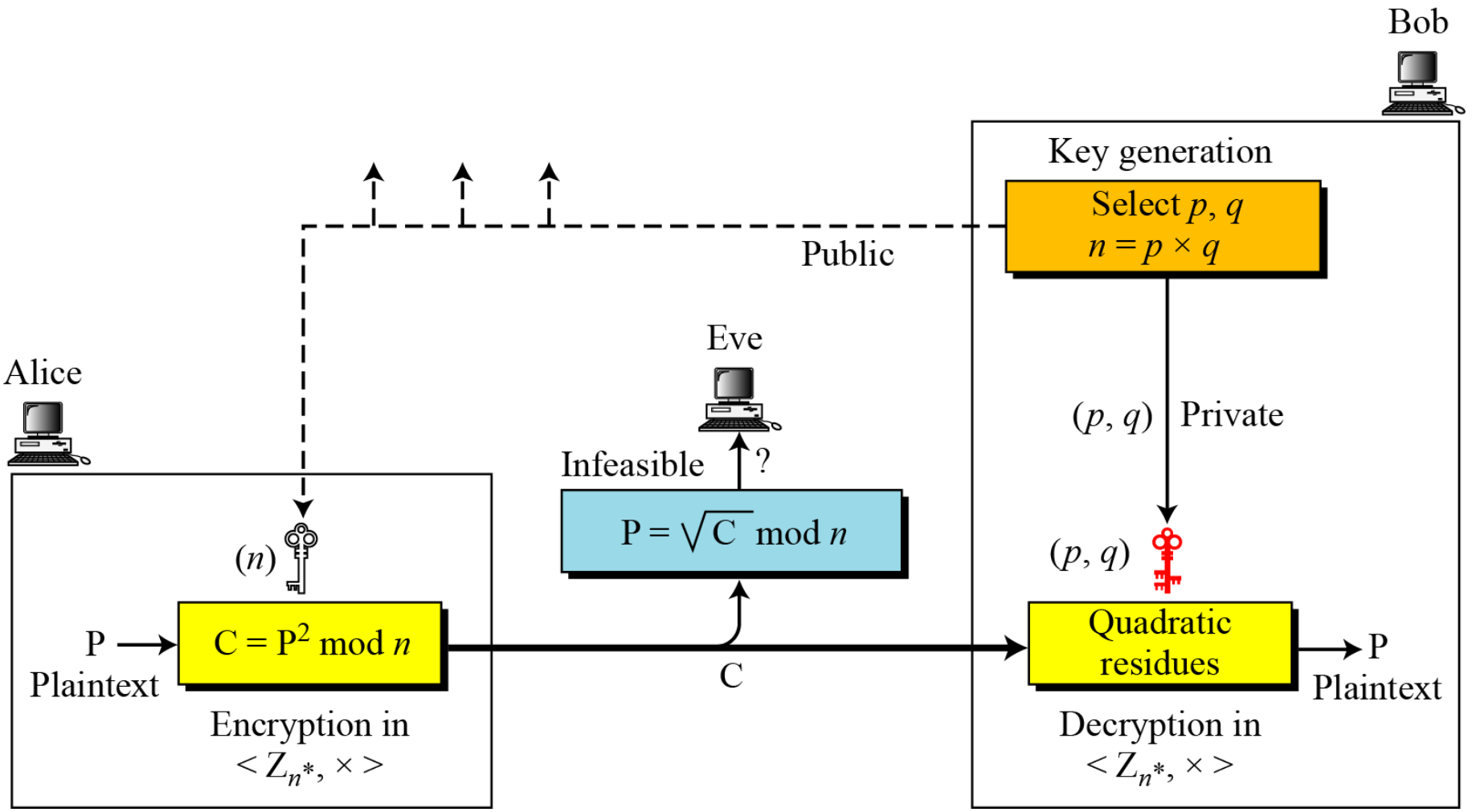
Decryption:

– Compute $P \equiv C^{1/2} \pmod{n}$

– There are four square roots m_1, m_2, m_3,m_4 of c modulo n.

– The message m is equal to one of these four messages

Figure 10.10 *Rabin cryptosystem*



10.3.1 Procedure

Key Generation

Algorithm 10.6 *Key generation for Rabin cryptosystem*

Rabin_Key_Generation

{

Choose two large primes p and q in the form $4k + 3$ and $p \neq q$.

$n \leftarrow p \times q$

Public_key $\leftarrow$ n // To be announced publicly

Private_key $\leftarrow$ (q, n) // To be kept secret

return Public_key and Private_key

}

Encryption

Algorithm 10.7 *Encryption in Rabin cryptosystem*

```
Rabin_Encryption (n, P)          // n is the public key; P is the ciphertext from Z_n*
{
    C ← P^2 mod n                // C is the ciphertext
    return C
}
```

Decryption

Algorithm 10.8 *Decryption in Rabin cryptosystem*

Rabin_Decryption (p, q, C) // C is the ciphertext; p and q are private keys

{

$a_1 \leftarrow +(C^{(p+1)/4}) \bmod p$

$a_2 \leftarrow -(C^{(p+1)/4}) \bmod p$

$b_1 \leftarrow +(C^{(q+1)/4}) \bmod q$

$b_2 \leftarrow -(C^{(q+1)/4}) \bmod q$

// The algorithm for the Chinese remainder algorithm is called four times.

$P_1 \leftarrow$ Chinese_Remainder (a_1, b_1, p, q)

$P_2 \leftarrow$ Chinese_Remainder (a_1, b_2, p, q)

$P_3 \leftarrow$ Chinese_Remainder (a_2, b_1, p, q)

$P_4 \leftarrow$ Chinese_Remainder (a_2, b_2, p, q)

return P_1, P_2, P_3, and P_4

}

Note

The Rabin cryptosystem is not deterministic: Decryption creates four plaintexts.

10.3.1 Continued

Example 10. 9

Here is a very trivial example to show the idea.

1. **Bob selects $p = 23$ and $q = 7$. Note that both are congruent to 3 mod 4.**
2. **Bob calculates $n = p \times q = 161$.**
3. **Bob announces n publicly; he keeps p and q private.**
4. **Alice wants to send the plaintext $P = 24$. Note that 161 and 24 are relatively prime; 24 is in Z_{161}^*.**
5. **She calculates $C = 24^2 = 93 \bmod 161$, and sends the ciphertext 93 to Bob.**

10.3.1 Continued

Example 10. 9

5. **Bob receives 93 and calculates four values:**

   $a_1 = +(93^{(23+1)/4}) \bmod 23 = 1 \bmod 23$

   $a_2 = -(93^{(23+1)/4}) \bmod 23 = 22 \bmod 23$

   $b_1 = +(93^{(7+1)/4}) \bmod 7 = 4 \bmod 7$

   $b_2 = -(93^{(7+1)/4}) \bmod 7 = 3 \bmod 7$

6. **Bob takes four possible answers,**

   (a_1, b_1), (a_1, b_2), (a_2, b_1), and (a_2, b_2),

   and uses the Chinese remainder theorem to find four possible plaintexts: 116, 24, 137, and 45.

5. **Note that only the second answer is Alice's plaintext.**

CHINESE REMAINDER THEOREM

It is possible to reconstruct integers in a certain range from their residues modulo, a set of pairwise relatively prime moduli.

A Simple Example of redundancy:

a real example would use primes in the range
from **512** to **1024** bits long, just as in the case of RSA.

Say, **p = 7**, **q = 11**, and **n = 77**.

Then **(-3)*7 + 2*11 = 1**, so that **a = -3** and **b = 2**.

Suppose one uses **3**-bit messages whose bits are then replicated to give **6** bits, up to the number **63**. Messages must be in the range from **1** to **70**, so this system of redundancy will work.

Start with data bits 101_2 or 5_{10}.

The replication gives 101101_2 or 45_{10}. Then $c = m^2 \bmod 77 = 23$.

Continuing the calculations, $r = 23^2 \bmod 7 = 4$, and $s = 23^3 \bmod 11 = 1$.

Finally, $x = ((-3)*7*1 + 2*11*4) \bmod 77 = 67$

And $y = ((-3)*7*1 - 2*11*4) \bmod 77 = 45$.

These are two of the four square roots, and the remaining two are $-x \bmod 77 = 10$ and $-y \bmod 77 = 32$. In binary, the four square roots are $67 = 1000011_2$, $45 = 101101_2$, $10 = 001010_2$, and $32 = 100000_2$.

Only **45** has the required redundancy, so this is the only number that this modified Rabin machine will return.

Effectiveness:

Decoding produces three false results in addition to the correct one, so that the correct result must be guessed. This is the major disadvantage of the Rabin cryptosystem and one of the factors which have prevented it from finding widespread practical use.

If the plaintext is intended to represent a text message, guessing is not difficult; however, if the plaintext is intended to represent a numerical value, this issue becomes a problem that must be resolved by some kind of disambiguation scheme. It is possible to choose plaintexts with special structures, or to add padding to eliminate this problem. The most common way of implementing disambiguation is known as Blum disambiguation.

Disambiguation introduces additional computational costs, and is what has prevented the Rabin cryptosystem from finding widespread practical use.

Security

The great advantage of the Rabin cryptosystem is that the code can only be broken if the codebreaker is capable of efficiently factoring the public key *n*.

It has been proven that decoding the Rabin cryptosystem is equivalent to the factorization problem, unlike in RSA.

Thus the Rabin system is more secure than RSA, and will remain so until a general solution for the factorization problem is discovered.

Discrete logarithm problem:

Let 'a' be a primitive root of a prime no. p, as one whose powers generate all integers from 1 to p-1.

i.e. $a \bmod p$,

$a^2 \bmod p$,

… …

$a^{p-1} \bmod p$.

are distinct and consists of integers from 1 to p-1.

For any integer b , we can find a unique exponent i

s.t. $b \equiv a^i \bmod p$

- exponent i is the discrete logarithm
- Ex: 2 is a primitive root of 11.

10-4 ELGAMAL CRYPTOSYSTEM

Besides RSA and Rabin,
another public-key cryptosystem is ElGamal.

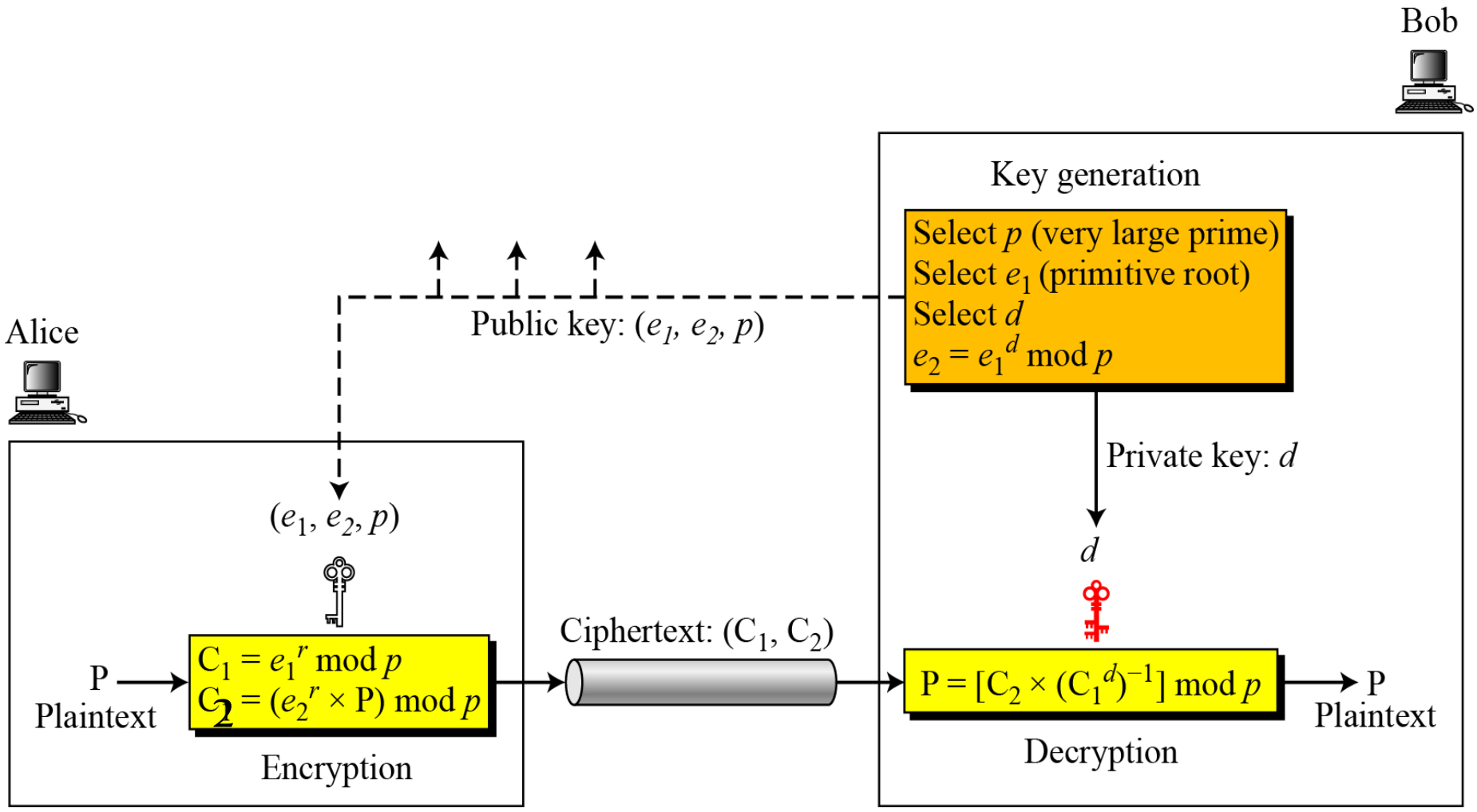
ElGamal is based on the discrete logarithm problem.

i.e. given e2, e1 and p , it is infeasible to calculate

$$r = \log_{e1} e_2 \bmod p$$

10.4.2 Procedure

Figure 10.11 *Key generation, encryption, and decryption in ElGamal*



Key Generation

Algorithm 10.9 *ElGamal key generation*

ElGamal_Key_Generation

{

Select a large prime p

Select d to be a member of the group $\mathbf{G} = < \mathbf{Z}_p^*, \times >$ such that $1 \leq d \leq p - 2$

Select e_1 to be a primitive root in the group $\mathbf{G} = < \mathbf{Z}_p^*, \times >$

$e_2 \leftarrow e_1^{\,d} \bmod p$

Public_key $\leftarrow (e_1, e_2, p)$ // To be announced publicly

Private_key $\leftarrow d$ // To be kept secret

return Public_key and Private_key

}

10.4.2 Continued

Algorithm 10.10 *ElGamal encryption*

```
ElGamal_Encryption (e1, e2, p, P)          // P is the plaintext
{
    Select a random integer r in the group G = < Zp*, × >
    C1  ← e1^r mod p
    C2  ← (P × e2^r) mod p                 // C1 and C2 are the ciphertexts
    return C1 and C2
}
```

10.4.2 Continued

Algorithm 10.11 *ElGamal decryption*

```
ElGamal_Decryption (d, p, C1, C2)          // C1 and C2 are the ciphertexts
{
    P ← [C2 (C1^d)^−1] mod p               // P is the plaintext
    return P
}
```

Note

The bit-operation complexity of encryption or decryption in ElGamal cryptosystem is polynomial.

10.4.3 Continued

Example 10. 10

Here is a trivial example. Bob chooses p = 11 and $e_1 = 2$. and d = 3 $e_2 = e_1^d = 8$. So the public keys are (2, 8, 11) and the private key is 3. Alice chooses r = 4 and calculates C1 and C2 for the plaintext 7.

Plaintext: 7

$$C_1 = e_1^r \bmod 11 = 16 \bmod 11 = 5 \bmod 11$$

$$C_2 = (P \times e_2^r) \bmod 11 = (7 \times 4096) \bmod 11 = 6 \bmod 11$$

Ciphertext: (5, 6)

Bob receives the ciphertexts (5 and 6) and calculates the plaintext.

$$[C_2 \times (C_1^d)^{-1}] \bmod 11 = 6 \times (5^3)^{-1} \bmod 11 = 6 \times 3 \bmod 11 = 7 \bmod 11$$

Plaintext: 7

10.4.3 Continued

Example 10. 11

Instead of using $P = [C_2 \times (C_1^d)^{-1}] \bmod p$ for decryption, we can avoid the calculation of multiplicative inverse and use $P = [C_2 \times C_1^{p-1-d}] \bmod p$ (see Fermat's little theorem in Chapter 9). In Example 10.10, we can calculate $P = [6 \times 5^{11-1-3}] \bmod 11 = 7 \bmod 11$.

Note

For the ElGamal cryptosystem, *p* must be at least 300 digits and *r* must be new for each encipherment.

10.4.3 Continued

Example 10. 12

Bob uses a random integer of 512 bits. The integer p is a 155-digit number (the ideal is 300 digits). Bob then chooses e_1, d, and calculates e_2, as shown below:

p =	11534899272561676244925313717014331740490094532609834959814346921905689869862264593212975473787189514436889176526473093615929993728061165964347353440008577
e_1 =	2
d =	1007
e_2 =	97886413043009189508766856938097739043880062887337687610022062233255450707415618921231831770461014167336015088413294085724853770315820660100725587074 55

10.4.3 Continued

Example 10. 10

Alice has the plaintext P = 3200 to send to Bob. She chooses r = 545131, calculates C1 and C2, and sends them to Bob.

P =	3200
***r* =**	545131
C_1 =	887297069383528471022570471492275663120260067256562125018188351429417223599712681114105363661705173051581533189165400973736355080295736788569060619152881
C_2 =	708454333048929944577016012380794999567436021836192446961774506921244696155165800779455593080345889614402408599525919579209721628879681350582779566430295O

Bob calculates the plaintext $P = C_2 \times ((C_1)^d)^{-1} \bmod p = 3200 \bmod p$.

P =	3200

10-5 ELLIPTIC CURVE CRYPTOSYSTEMS

Although RSA and ElGamal are secure asymmetric-key cryptosystems, their security comes with a price, their large keys. Researchers have looked for alternatives that give the same level of security with smaller key sizes. One of these promising alternatives is the elliptic curve cryptosystem (ECC).

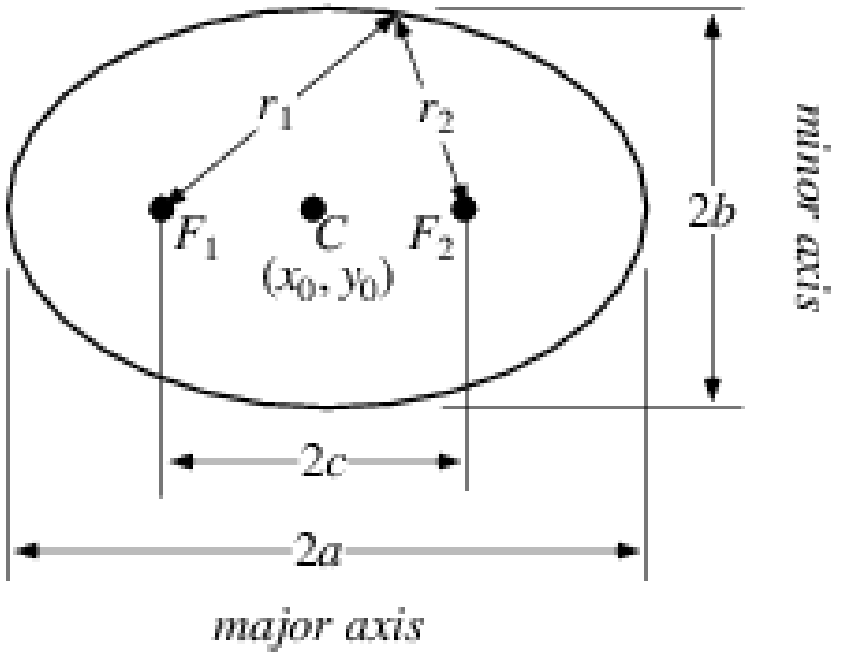
Topics discussed in this section:

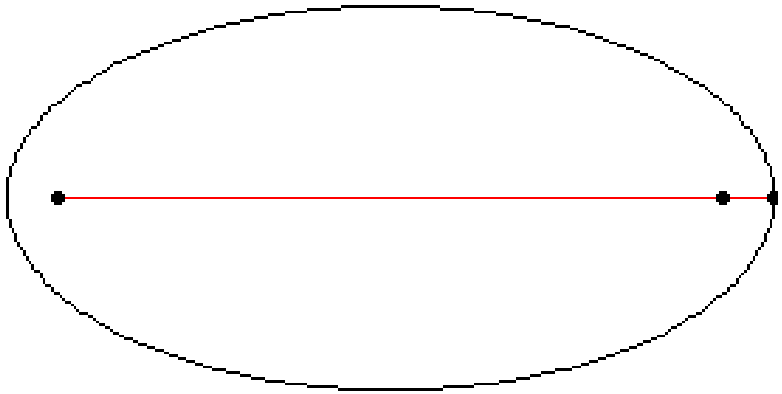
10.5.1 Elliptic Curves over Real Numbers
10.5.2 Elliptic Curves over GF(*p*)
10.5.3 Elliptic Curves over GF(2^n)
10.5.4 Elliptic Curve Cryptography Simulating ElGamal

Ellipse

The equation of an ellipse in Cartesian coordinates is:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1.$$





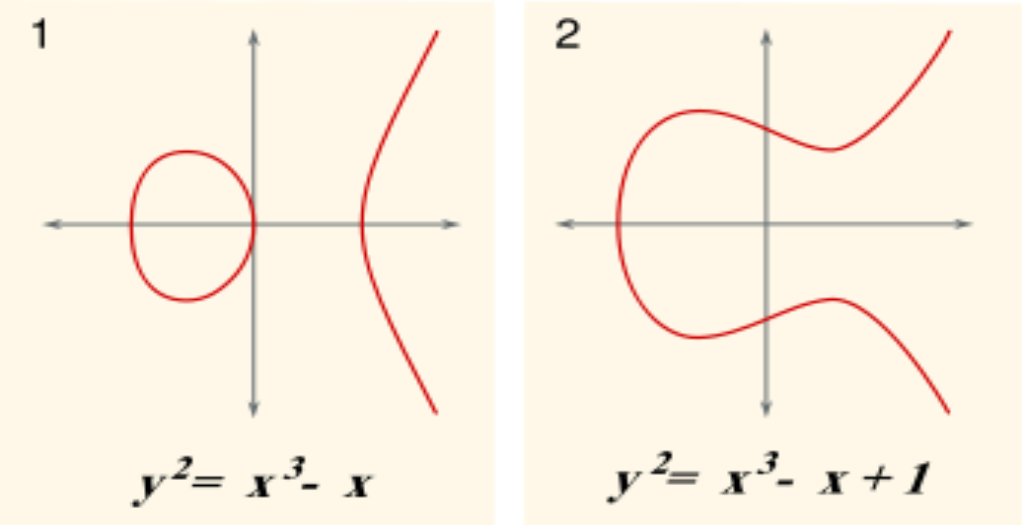
10.5.1 Elliptic Curves over Real Numbers

An *elliptic curve* is a plane curve which consists of the points satisfying the equation. ***The general equation for an elliptic curve is***

$$y^2 + b_1xy + b_2y = x^3 + a_1x^2 + a_2x + a_3$$

Elliptic curves over real numbers use a special class of elliptic curves of the form

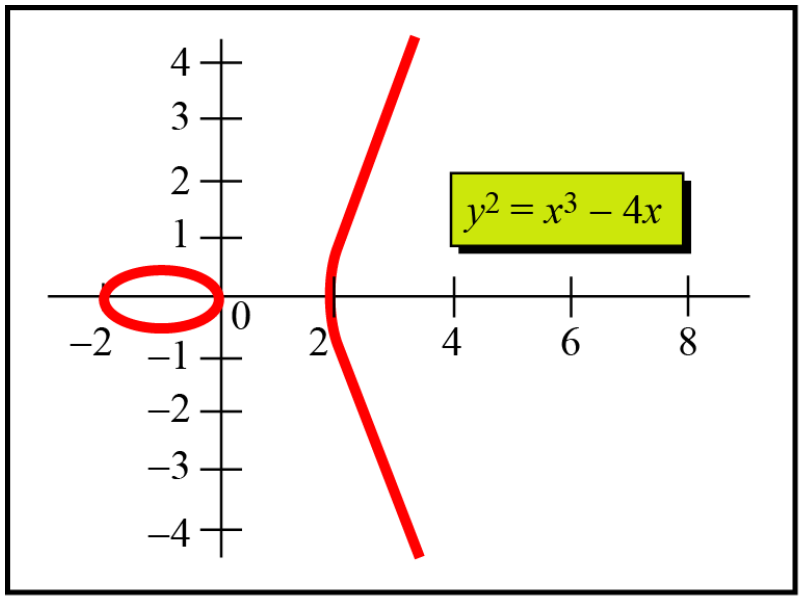
$$y^2 = x^3 + ax + b$$



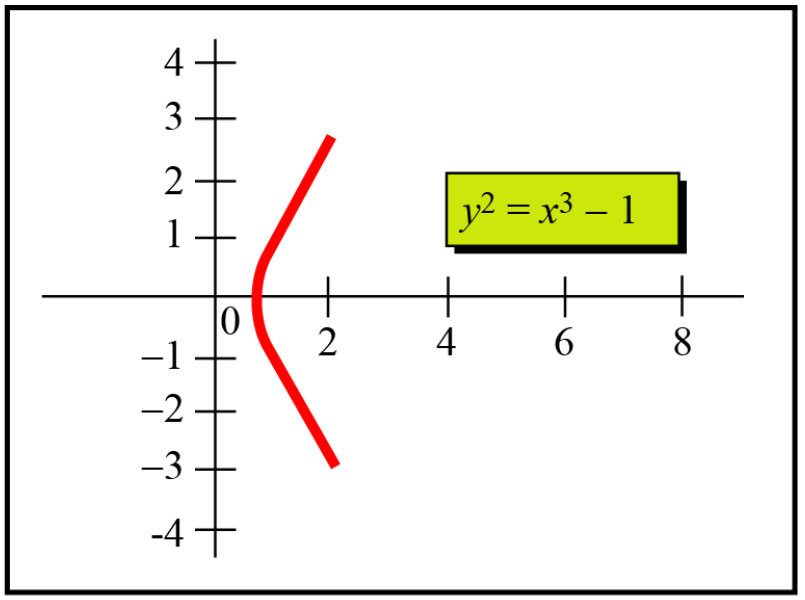
Example 10. 13

Figure 10.12 shows two elliptic curves with equations $y^2 = x^3 - 4x$ and $y^2 = x^3 - 1$. Both are nonsingular. However, the first has three real roots ($x = -2$, $x = 0$, and $x = 2$), but the second has only one real root ($x = 1$) and two imaginary ones.

Figure 10.12 *Two elliptic curves over a real field*



a. Three real roots



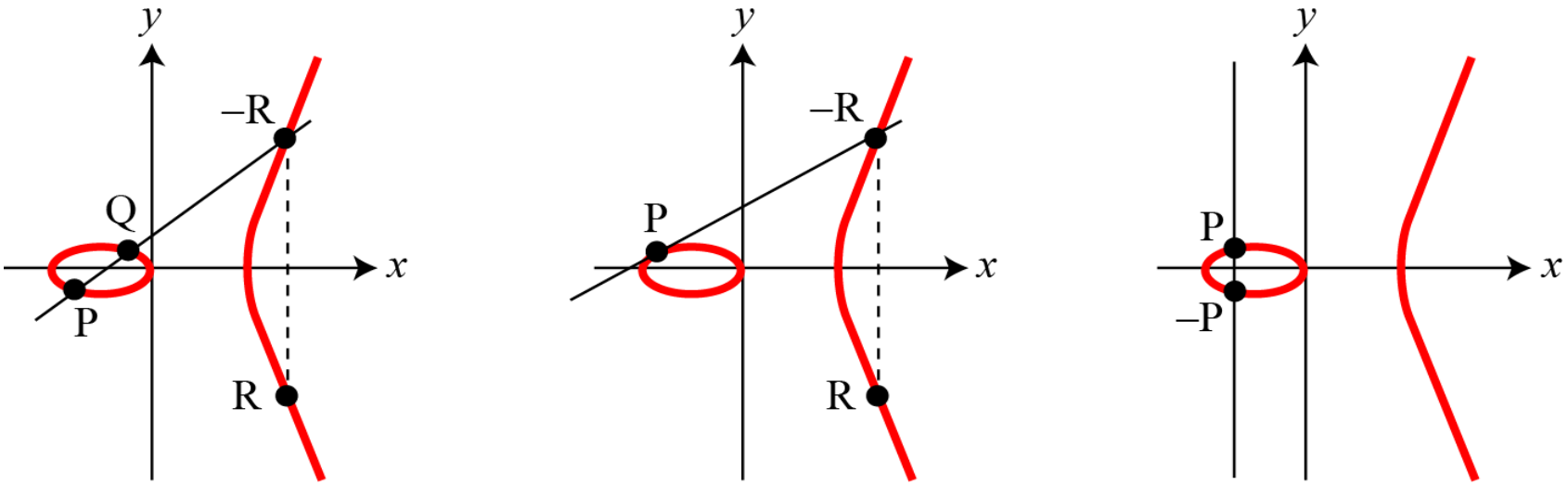
b. One real and two imaginary roots

An Abelian Group

Define a point $P=(x_1,y_1)$ on the elliptic curve and a group of such points, $Q=(x_2,y_2)$, $R=(x_3,y_3)$ etc. forms an Abelian Group that allows *addition* operation.

i.e. $G = \langle E,+ \rangle$ Where $E=\{ (2.0,0.0), (0.0,0.0), (-2.0,0.0), (10.0,30.98)$

Figure 10.13 ***Three adding cases in an elliptic curve***



a. (R = P + Q) b. (R = P + P) c. (O = P + (– P))

1. The line connecting P and Q intercepts the curve at –R.
   R is the reflection of –R.

10.5.1 Continued

The slope of the line λ

1.

$$\lambda = (y_2 - y_1) / (x_2 - x_1)$$

$$x_3 = \lambda^2 - x_1 - x_2 \qquad y_3 = \lambda (x_1 - x_3) - y_1$$

2.

$$\lambda = (3x_1^2 + a)/(2y_1)$$

$$x_3 = \lambda^2 - x_1 - x_2 \qquad y_3 = \lambda (x_1 - x_3) - y_1$$

3. The intercepting point is at infinity; a point O as the point at infinity or zero point, which is the additive identity of the group.

Curves and Key Length

ECC keys have length, which directly depends on the underlying curve. In most applications (like OpenSSL, OpenSSH and Bitcoin) the default key length for the ECC private keys is 256 bits, but depending on the curve many different ECC key sizes are possible: 192-bit (curve **secp192r1**), 233-bit (curve **sect233k1**), 224-bit (curve **secp224k1**), 256-bit (curves **secp256k1** and **Curve25519**), 283-bit (curve **sect283k1**), 384-bit (curves **p384** and **secp384r1**), 409-bit (curve **sect409r1**), 414-bit (curve **Curve41417**), 448-bit (curve **Curve448-Goldilocks**), 511-bit (curve **M-511**), 521-bit (curve **P-521**), 571-bit (curve **sect571k1**) and many others.

ECC Algorithms

Elliptic-curve cryptography (ECC) provides several groups of algorithms, based on the math of the elliptic curves over finite fields:

- ECC digital signature algorithms like ECDSA (for classical curves) and EdDSA (for twisted Edwards curves).
- ECC encryption algorithms and hybrid encryption schemes like the ECIES integrated encryption scheme and EEECC (EC-based ElGamal).
- ECC key agreement algorithms like ECDH, X25519 and FHMQV.

Security rely upon the difficulty of the ECDLP (elliptic curve discrete logarithm problem).

- All these algorithms use public / private key pairs, where the private key is an integer and the public key is a point on the elliptic curve (EC point).

In ECC we have **E**lliptic curve (EC) over finite field $\mathbb{F}p$

- G = generator point (fixed constant, a base point on the EC)
- k = private key (integer)
- P = public key (point)

- $P = k * G$ (very fast to calculate)
- $k = P / G$ (extremely slow, considered infeasible for large k) to calculate)
- known as the ECDLP problem

ECC and Safe Curve

Eq -> y^2 = x^3 + ax + b

NIST P-256	y^2 = x^3 - 3x + 41058363725152142129326129780047268409114441015993725554835256314039467401291
Base Point	(48439561293906451759052585252797914202762949526041747995844080717082404635286361342509567497957985851279195878819566111066729850150718771982535684144051 09) = (0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c2960x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5)

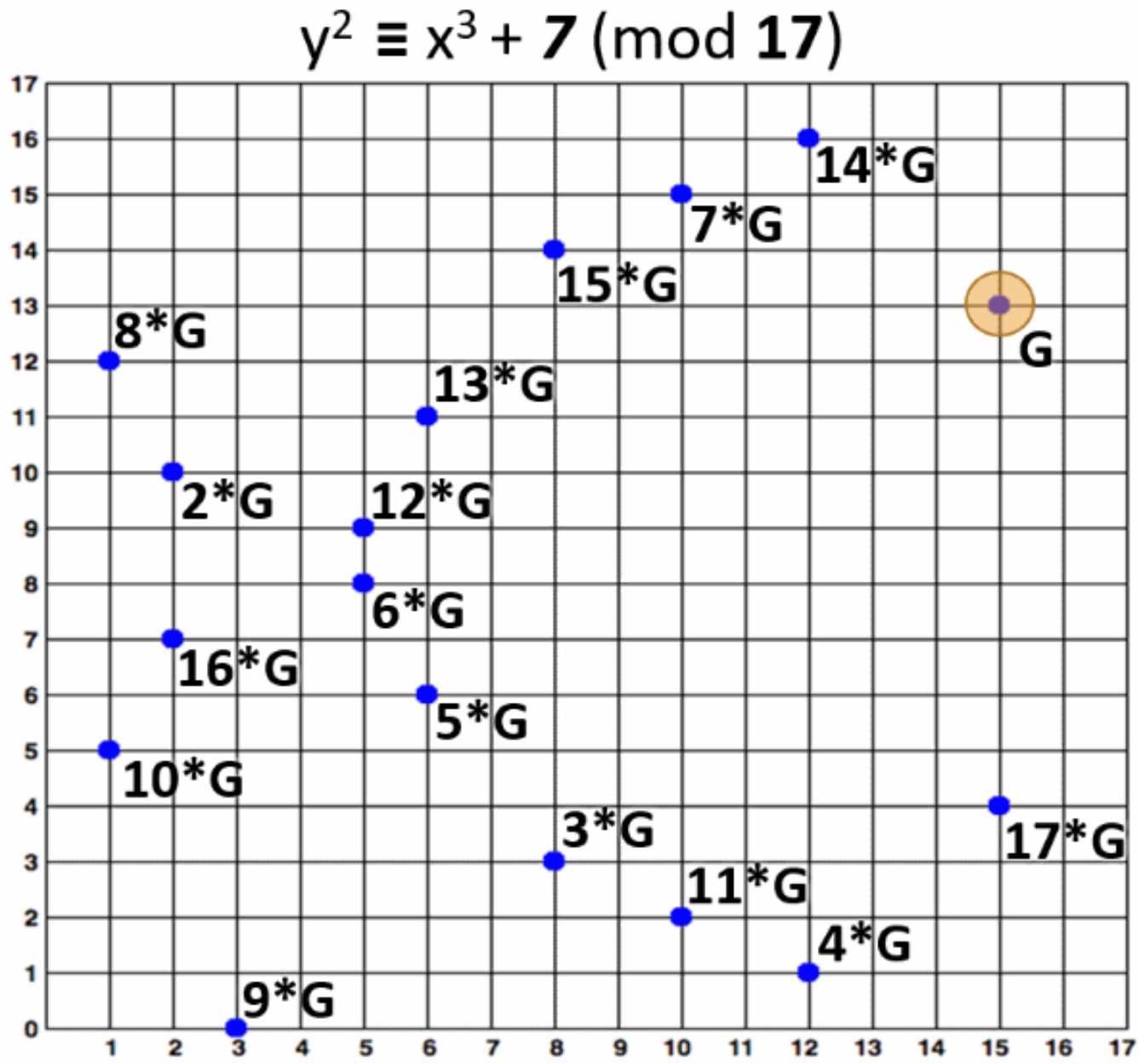
Multiplication of EC Points - Example in Python

use the Python library tinyec

pip install tinyec

Example $y2 \equiv x3 + 7 \pmod{17}$

```
from tinyec.ec import SubGroup, Curve
field = SubGroup(p=17, g=(15, 13), n=18, h=1)
curve = Curve(a=0, b=7, field=field, name='p1707')
print('curve:', curve)
for k in range(0, 25):
p = k * curve.g
print(f"{k} * G = ({p.x}, {p.y})")
```


$y^2 \equiv x^3 + 7 \pmod{17}$
14*G
7*G
15*G
G
8*G
13*G
2*G
12*G
6*G
16*G
5*G
10*G
3*G
17*G
11*G
4*G
9*G

10.5.2 Elliptic Curves over GF(p) (Galois Fields)

Define an elliptic curve Group with addition operation over the GF(p) Field as Ep(a, b) where p defines the modulus and a and b the coefficients of the equation

$$y^2=x^3+a.x+b$$

The field defines the add, sub, mult, division operations on real numbers that are needed to find the addition of the points in the group.

Finding an Inverse

The inverse of a point (x, y) is (x, −y), where −y is the additive inverse of y. For example, if p = 13, the inverse of (4, 2) is (4, 11).

Finding Points on the Curve

Algorithm 10.12 shows the pseudocode for finding the points on the curve Ep(a, b).

$$y^2 \equiv x^3 + a.x + b \pmod{p}.$$

Algorithm 10.12 *Pseudocode for finding points on an elliptic curve*

```
ellipticCurve_points (p, a, b)                    // p is the modulus
{
    x ← 0
    while (x < p)
    {
        w ← (x^3 + ax + b) mod p                  // w is y^2
        if (w is a perfect square in Z_p) output (x, √w) (x, − √w)
        x ← x + 1
    {
}
```

Example 10. 14

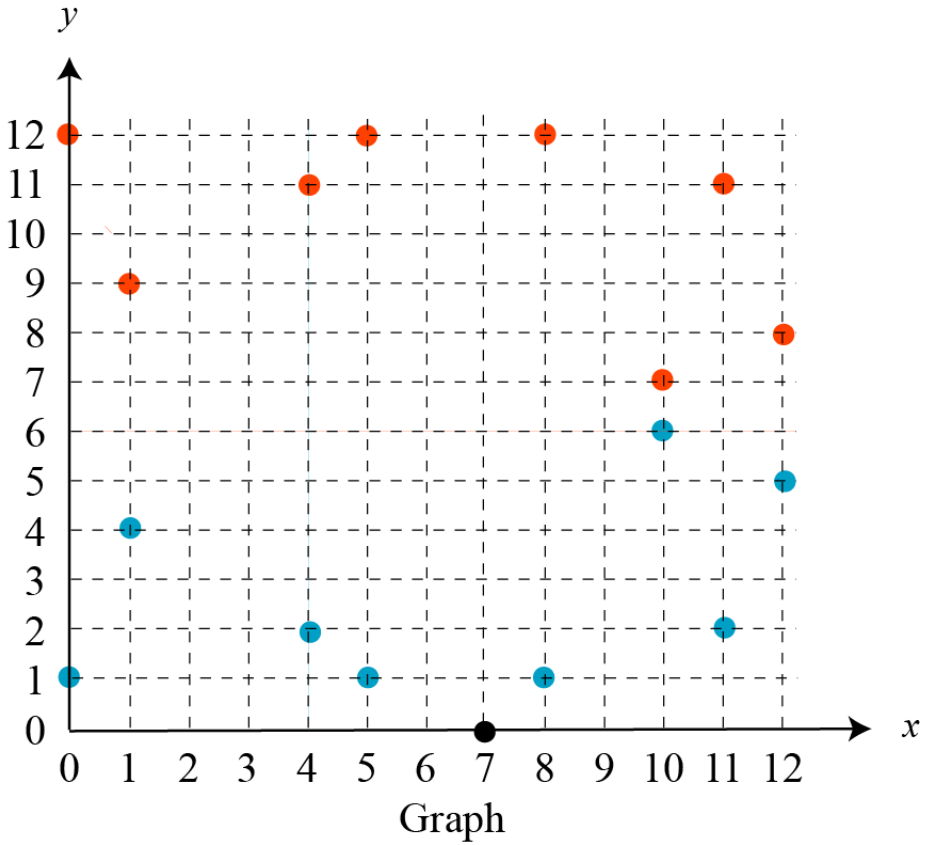
Ex: $E_{13}(1,1)$ ***i.e. The equation is*** $y^2 = x^3 + x + 1$ ***and the calculation is done modulo 13.***

Figure 10.14 *Points on an elliptic curve over GF(p)*

P	–P
(0, 1)	(0, 12)
(1, 4)	(1, 9)
(4, 2)	(4, 11)
(5, 1)	(5, 12)
(7, 0)	(7, 0)
(8, 1)	(8, 12)
(10, 6)	(10, 7)
(11, 2)	(11, 11)
(12, 5)	(12, 18)

P = –P →

Points



10.5.2 Continued

Example 10. 15

Let us add two points in Example 10.14, $R = P + Q$, where
$P = (4, 2)$ and $Q = (10, 6)$.

a. $\lambda = (6 - 2) \times (10 - 4)^{-1} \bmod 13 = 4 \times 6^{-1} \bmod 13 = 5 \bmod 13.$

b. $x = (5^2 - 4 - 10) \bmod 13 = 11 \bmod 13.$

c. $y = [5 (4 - 11) - 2] \bmod 13 = 2 \bmod 13.$

d. R = (11, 2), which is a point on the curve in Example 10.14.

10.5.3 Elliptic Curves over $GF(2^n)$

To define an elliptic curve over $GF(2^n)$ Field, (the elements of the field are the set of n-bit words) one needs to change the cubic equation. The common equation is

$$y^2 + xy = x^3 + ax^2 + b$$

10.5.3 Continued

Finding Inverses

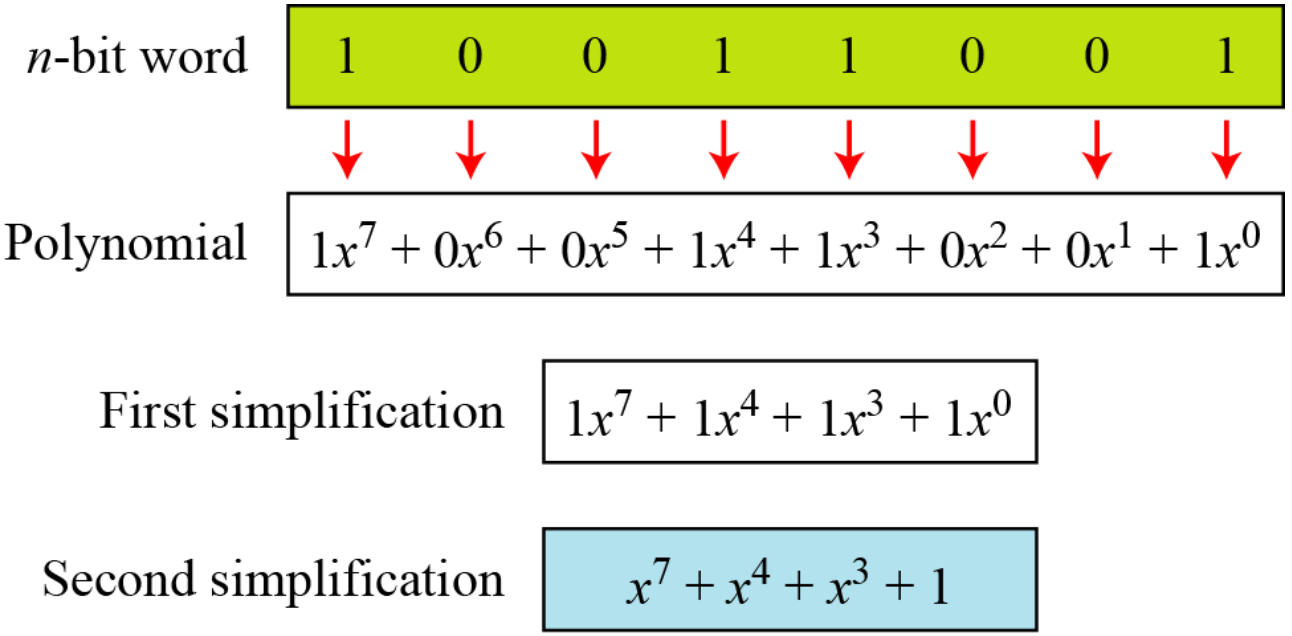
If $P = (x, y)$, then $-P = (x, x + y)$.

Finding Points on the Curve

We can write an algorithm to find the points on the curve using generators for polynomials discussed in Chapter 7. This algorithm is left as an exercise. Following is a very trivial example.

Figure 4.9 show how we can represent the 8-bit word (10011001) using a polynomials.

Figure 4.9 *Representation of an 8-bit word by a polynomial*



Example 4

Four cyclic subgroups can be made from the group $G = \langle Z6, + \rangle$. They are $H1 = \langle \{0\}, + \rangle$, $H2 = \langle \{0, 2, 4\}, + \rangle$, $H_3 = \langle \{0, 3\}, + \rangle$, and $H4 = G$.

$0^0 \bmod 6 = 0$

$1^0 \bmod 6 = 0$
$1^1 \bmod 6 = 1$
$1^2 \bmod 6 = (1 + 1) \bmod 6 = 2$
$1^3 \bmod 6 = (1 + 1 + 1) \bmod 6 = 3$
$1^4 \bmod 6 = (1 + 1 + 1 + 1) \bmod 6 = 4$
$1^5 \bmod 6 = (1 + 1 + 1 + 1 + 1) \bmod 6 = 5$

$2^0 \bmod 6 = 0$
$2^1 \bmod 6 = 2$
$2^2 \bmod 6 = (2 + 2) \bmod 6 = 4$

$3^0 \bmod 6 = 0$
$3^1 \bmod 6 = 3$

$4^0 \bmod 6 = 0$
$4^1 \bmod 6 = 4$
$4^2 \bmod 6 = (4 + 4) \bmod 6 = 2$

$5^0 \bmod 6 = 0$
$5^1 \bmod 6 = 5$
$5^2 \bmod 6 = 4$
$5^3 \bmod 6 = 3$
$5^4 \bmod 6 = 2$
$5^5 \bmod 6 = 1$

10.5.3 Continued

Example 10. 16

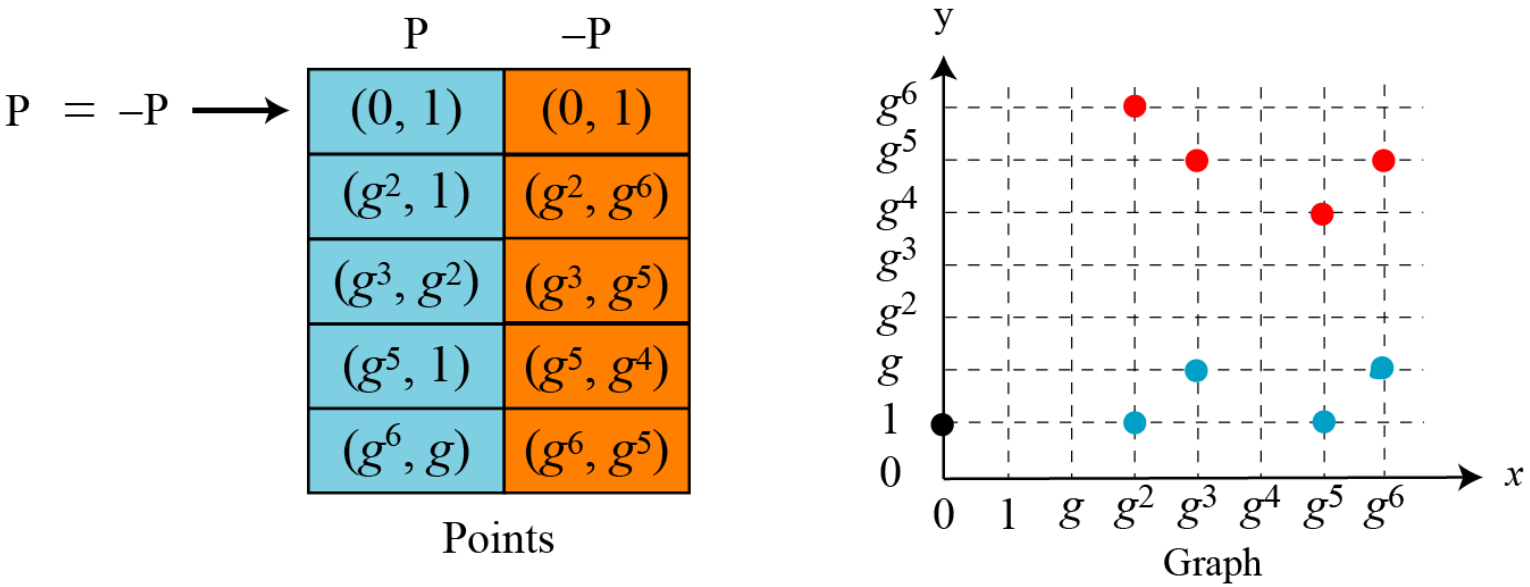
We choose GF(2^3) with elements {0, 1, g, g^2, g^3, g^4, g^5, g^6} using the irreducible polynomial of $f(x) = x^3 + x + 1$, which means that $g^3 + g + 1 = 0$ or $g^3 = g + 1$. Other powers of g can be calculated accordingly. The following shows the values of the g's.

0	000	$g^3 = g + 1$	011
1	001	$g^4 = g^2 + g$	110
g	010	$g^5 = g^2 + g + 1$	111
g^2	100	$g^6 = g^2 + 1$	101

Example 10. 16 *Continued*

Using the elliptic curve $y^2 + xy = x^3 + g^3x^2 + 1$, with $a = g^3$ and $b = 1$, we can find the points on this curve, as shown in Figure 10.15..

Figure 10.15 *Points on an elliptic curve over GF(2n)*



10.5.3 Continued

Adding Two Points

1. If $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $Q \neq -P$, and $Q \neq P$, then $R = (x_3, y_3) = P + Q$ can be found as

$$\lambda = (y_2 + y_1) / (x_2 + x_1)$$

$$x_3 = \lambda^2 + \lambda + x_1 + x_2 + a \qquad y_3 = \lambda (x_1 + x_3) + x_3 + y_1$$

If $Q = P$, then $R = P + P$ (or $R = 2P$) can be found as

$$\lambda = x_1 + y_1 / x_1$$

$$x_3 = \lambda^2 + \lambda + a \qquad y_3 = x_1^2 + (\lambda + 1) x_3$$

10.5.3 Continued

Example 10. 17

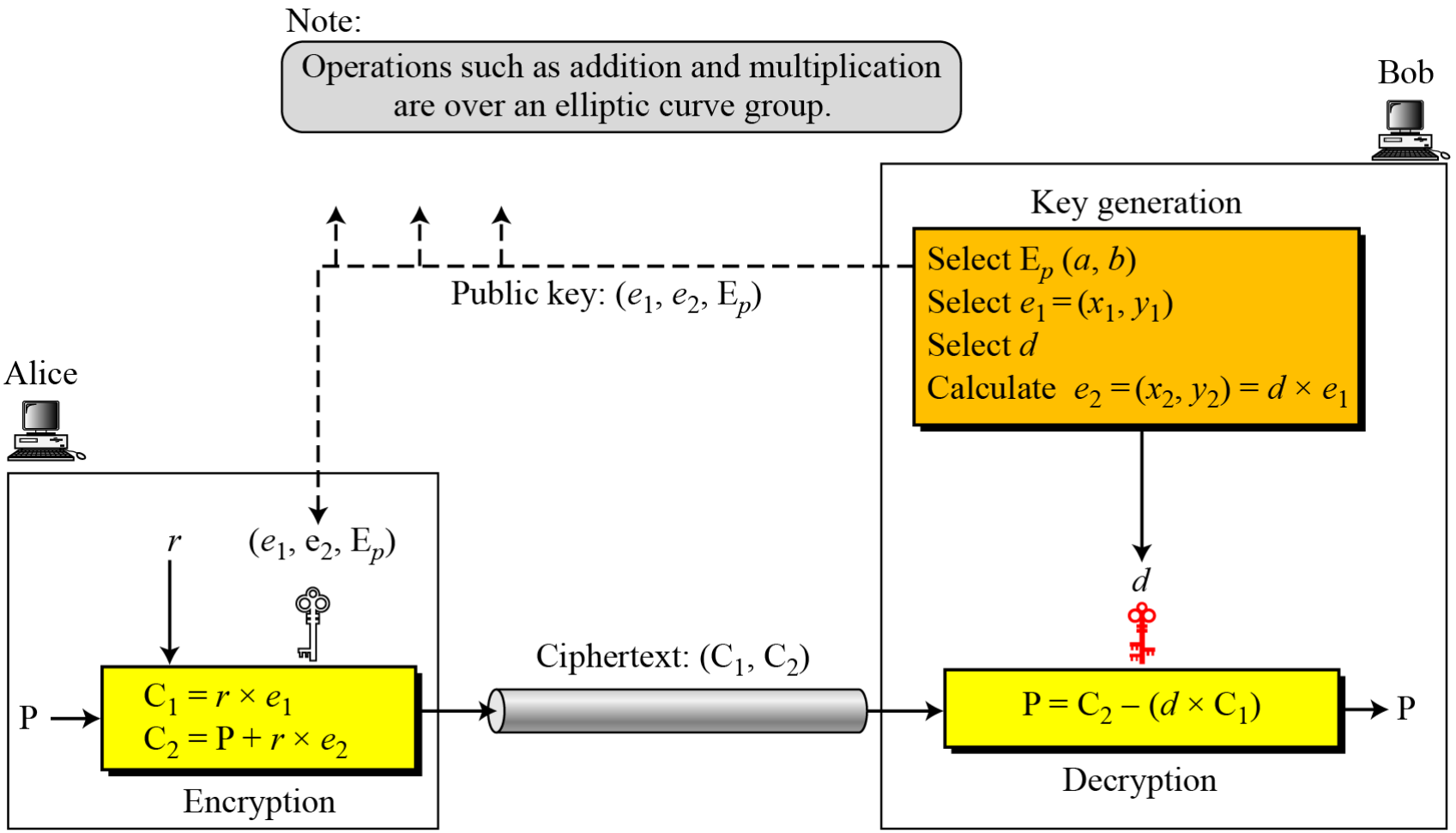
Let us find $R = P + Q$, where $P = (0, 1)$ and $Q = (g^2, 1)$.
We have $\lambda = 0$ and $R = (g^5, g^4)$.

Example 10. 18

Let us find $R = 2P$, where $P = (g^2, 1)$. We have $\lambda = g^2 + 1/g^2 = g^2 + g^5 = g + 1$ and $R = (g^6, g^5)$.

10.5.4 ECC Simulating ElGamal

Figure 10.16 *ElGamal cryptosystem using the elliptic curve*



10.5.4 Continued

Generating Public and Private Keys

$E(a, b)$ $e_1(x_1, y_1)$ d $e_2(x_2, y_2) = d \times e_1(x_1, y_1)$

Encryption

$$C_1 = r \times e_1 \qquad C_2 = P + r \times e_2$$

Decryption

$$P = C_2 - (d \times C_1)$$

The minus sign here means adding with the inverse.

Note

The security of ECC depends on the difficulty of solving the elliptic curve logarithm problem.

10.5.4 Continued

Example 10. 19

Here is a very trivial example of encipherment using an elliptic curve over GF(p).

1. *Bob selects $E_{67}(2, 3)$ as the elliptic curve over GF(p).*
2. *Bob selects $e_1 = (2, 22)$ and $d = 4$.*
3. *Bob calculates $e_2 = (13, 45)$, where $e_2 = d \times e_1$.*
4. *Bob publicly announces the tuple (E, e_1, e_2).*
5. *Alice wants to send the plaintext $P = (24, 26)$ to Bob. She selects $r = 2$.*
6. *Alice finds the point $C_1 = (35,1)$, where $C_1 = r \times e_1$.*

10.5.4 Continued…

7. Alice finds the point $C_2=(21,44)$, where $C_2=P+r \times e_2$
8. Bob receives C_1 and C_2 . He uses $2 \times C_1(35,1)$ to get (23,25)
9. Bob inverts the point (23,25) to get the point (23, 42)
10. Bob adds (23,42) with $C_2=(21,44)$ to get the original plaintext P= (24,26)

Security of ECC

1. The security of ECC depends on the difficulty of solving the Elliptic Curve Discrete Logarithmic Problem (ECDLP).
2. Value of r and d are secret.

Elliptic Curve Groups over real numbers

QUIZ:

1. Does the elliptic curve equation $y^2 = x^3 - 7x - 6$ over real numbers define a group?

2. What is the additive identity of regular integers?

3. Is (4,7) a point on the elliptic curve $y^2 = x^3 - 5x + 5$ over real numbers?

4. What are the negatives of the following elliptic curve points over real numbers?
P(-4,-6), Q(17,0), R(3,9), S(0,-4)

5. In the elliptic curve group defined by $y^2 = x^3 - 17x + 16$ over real numbers, what is P + Q if P = (0,-4) and Q = (1,0)?

6. In the elliptic curve group defined by $y^2 = x^3 - 17x + 16$ over real numbers, what is 2P if P = (4, 3.464)?

QUIZ - Solutions

1. Does the elliptic curve equation $y^2 = x^3 - 7x - 6$ over real numbers define a group?

Yes, since $4a^3 + 27b^2 = 4(-7)^3 + 27(-6)^2 = -400$

The equation $y^2 = x^3 - 7x - 6$ does define an elliptic curve group because $4a^3 + 27b^2$ is not 0.

2. What is the additive identity of regular integers?

The additive identity of regular integers is 0, since $x + 0 = x$ for all integers.

3. Is (4,7) a point on the elliptic curve $y^2 = x^3 - 5x + 5$ over real numbers?

Yes, since the equation holds true for $x = 4$ and $y = 7$:

$$(7)^2 = (4)^3 - 5(4) + 5$$
$$49 = 64 - 20 + 5$$
$$49 = 49$$

4. What are the negatives of the following elliptic curve points over real numbers?

P(-4,-6), Q(17,0), R(3,9), S(0,-4)

The negative is the point reflected through the x-axis. Thus

-P(-4,6), -Q(17,0), -R(3,-9), -S(0,4)

5. In the elliptic curve group defined by $y^2 = x^3 - 17x + 16$ over real numbers, what is P + Q if P = (0,-4) and Q = (1,0)?

From the Addition formulae:

$s = (y_P - y_Q) / (x_P - x_Q) = (-4 - 0) / (0 - 1) = 4$

$x_R = s^2 - x_P - x_Q = 16 - 0 - 1 = 15$

and

$y_R = -y_P + s(x_P - x_R) = 4 + 4(0 - 15) = -56$

Thus P + Q = (15, -56)

6. In the elliptic curve group defined by $y^2 = x^3 - 17x + 16$ over real numbers, what is $2P$ if $P = (4, 3.464)$?

From the Doubling formulae:

$$s = (3x_P^2 + a) / (2y_P) = (3*(4)^2 + (-17)) / 2*(3.464) = 31 / 6.928 = 4.475$$

$$x_R = s^2 - 2x_P = (4.475)^2 - 2(4) = 20.022 - 8 = 12.022$$

and

$$y_R = -y_P + s(x_P - x_R) = -3.464 + 4.475(4 - 12.022) = -3.464 - 35.898 = -39.362$$

Thus $2P = (12.022, -39.362)$